

Artificial Intelligence Project Part B

Authors:

Mitchell Pooley - 1460127

Jiawei Cai - 1532072

Our Approach:

Our agent uses a hybrid strategy that switches between two search algorithms depending on the stage of the game. In the early and mid-game, we use iterative-deepening alpha-beta pruning with a static evaluation function. Once the game reaches turn 130, we switch to Monte Carlo Tree Search (MCTS) with a greedy-then-random rollout policy. This design was motivated by empirical testing: we found that alpha-beta's deterministic look-ahead performed strongest in the early stages of the game when the board is relatively structured, while MCTS handles the noisier mid-to-late game better due to its ability to explore a wider set of playouts under a fixed time budget. Arbitrary parameters are then tweaked by a self playing hill-climb python script that mutates the parameters and evolves the agent.

Placement Phase

During the placement phase, placements are chosen using a heuristic that scores candidate cells on three factors: proximity to the board centre (maximising future mobility) and proximity to our own existing stacks (encouraging merging for taller stacks early on) and an edge penalty to avoid edge and corner placements that are very dangerous and restrict movement options.

Early Game: Alpha-Beta Pruning

The agent uses iterative-deepening alpha-beta, searching from depth 1 up to a maximum depth, always returning the best action from the deepest fully completed iteration before the time deadline. Alpha-beta extends minimax search by pruning branches that cannot influence the final decision, significantly reducing the number of nodes evaluated without changing the result. The agent acts as the maximising player and assumes the opponent minimises the same evaluation function.

Move ordering is applied at every node to maximise pruning efficiency. Actions are sorted in descending priority: EatActions are tried first (scored at target height + large constant), followed by CascadeActions that hit enemy tokens along their path (scored at total enemy tokens hit + moderate constant), then MoveActions that merge with a friendly stack (scored at destination height + small constant), with all other moves scored at zero. Trying the most forcing moves first increases the likelihood of early beta cutoffs, substantially reducing the effective branching factor.

The static evaluation function used at leaf nodes scores the board on five features, returning a value in $[-1, 1]$:

- **Token ratio:** the difference in total tokens normalised by the total on the board. This rewards decreasing the number of enemy tokens while maximising the number of friendly tokens.
- **Eat threat balance:** the number of adjacent enemy stacks the agent can immediately capture minus the number of own stacks currently exposed to capture, normalised by total stack count. This rewards threatening positions and penalises threatened positions.
- **Tallest stack advantage:** the difference between the agent's tallest stack and the opponent's tallest stack, normalised by their sum. Tall stacks can eat more targets and cascade further, making them disproportionately valuable.
- **Centre control:** the average proximity of each player's stacks to the board centre, expressed as a difference. Central stacks have more adjacent cells and therefore more options.
- **Distance to closest enemy:** During late game, stacks are usually far apart, a number of moves must be completed in succession to successfully eliminate an enemy stack, Alpha-Beta does worse as the number of moves needed is larger than the maximum depth. This feature helps by encouraging stacks to close in on enemies, so that the optimal sequence of moves can be captured by the algorithm.

Weights shift dynamically as the game progresses, to reflect the changing importance of features as the game progresses. For example, distance to the closest enemy and the token ratio matters far more than centre control as the game progresses.

Late Game: MCTS with Random Playouts

After a specific turn count, the agent switches to MCTS. Each call to action runs selection, expansion, simulation, and backpropagation within a per-turn time budget computed as a fraction of remaining CPU time.

Selection uses UCT (Upper Confidence Bound for Trees), balancing exploitation of high-value nodes with exploration of less-visited ones.

Expansion adds one unvisited child per iteration. Actions are prioritised by type: EatActions are tried first (immediate captures are nearly always beneficial), followed by CascadeActions, then MoveActions.

A max rollout depth was added to the simulations as full games can run up to 200 turns usually, the end state after the max depth is then evaluated using the a similar evaluation method as the alpha-beta algorithm, with the difference being that centre control and distance to the nearest enemy are valued significantly less as in the late game, only a few pieces will be left and everything is simplified, complicated tactical move sequences do not exist anymore, so centre control is not needed. Additionally, distance to the nearest enemy has already been achieved by the previous turns so it's also excluded in the MCTS evaluation.

Backpropagation propagates the reward up the tree. Since turns alternate, the reward is inverted at each level so that each node accumulates value from the perspective of the player who moved to reach it. This ensures UCT selection correctly prefers nodes that are good for the current mover at each level.

The best action is then selected by most visits rather than the highest average value, which is more robust to outlier simulations.

Tree reuse is applied across turns: rather than rebuilding the search tree from scratch each turn, the root is advanced to the child node corresponding to the action actually played (by either player). This allows the agent to inherit previously accumulated visit counts and carry exploration forward. If the play action was never expanded, the tree is discarded and rebuilt fresh.

Performance Evaluation:

We evaluated five agent configurations against each other over many games, with each agent playing as RED and BLUE in equal proportion:

- **Greedy:** selects the action with the highest immediate static evaluation, with no look-ahead.
- **MCTS (Complex):** MCTS with a playout strategy that involves a complex move selection heuristic.
- **MCTS (random playouts):** MCTS with the greedy-eat-then-random rollout policy described above, no alpha-beta.
- **Alpha-Beta:** Iterative-deepening alpha-beta as described above, no MCTS.
- **Hybrid (submitted):** iterative-deepening alpha-beta for early/mid game, switching to MCTS for the endgame.

The hybrid agent outperformed the other approaches in head-to-head testing. The greedy approach performed poorly in most cases, and the complex MCTS struggled due to the strict time limit per game. Alpha-beta alone struggled in the late game, where the branching factor makes deep search expensive. MCTS alone with random rollouts, was weaker in the early game, where random simulations provide a poor signal in structured tactical positions. Combining both, using alpha-beta while the board is structured and switching to MCTS when the game simplifies, produced the strongest overall results.

Parameter Hill-Climbing:

All arbitrary parameters were tuned using a hill-climbing script with random mutation, operating as follows:

1. The current best parameters are taken as the base.
2. A random subset of parameters is perturbed by a small percentage to produce a mutant agent.
3. The base and mutant agents play four games against each other, alternating colours.
4. If the mutant wins at least three of the four games, its parameters replace the base.
5. Steps 1–4 are repeated for as many generations as time permits.

The script was run for approximately 20 hours, producing around 100 generations of mutants.

Only a random subset of parameters is mutated per generation rather than all of them simultaneously. Mutating all parameters at once increases the likelihood that beneficial and detrimental changes become entangled, making it difficult to attribute any improvement or degradation to a specific parameter. Mutating a subset allows beneficial changes to be identified and retained more reliably.

All mutations were deliberately kept small. Large mutations risk discarding the structural intentions of the original parameters — for example, that centre control should be weighted less heavily in the late game. Keeping mutations small also improves generalisation: an agent optimised too aggressively against itself may overfit to its own tendencies rather than performing well against agents using different strategies.

A threshold of three wins out of four was chosen as a balance between statistical confidence and computational cost. Given that parameters change only slightly per generation, an incorrect update is unlikely to cause significant degradation, and the overall change in performance across many generations remains upward. The hill-climbing process will eventually converge to a local maximum; with the time available, further improvement would likely require additional running time or a broader search strategy such as random restarts.

Other Technical Aspects:

Time management: rather than allocating a fixed time per turn, the agent spends a fixed fraction of remaining CPU time per turn, clamped between a minimum and a maximum number of seconds. This naturally front-loads computation early in the game when alpha-beta trees are shallow, and iterations are cheap, while conserving budget for the later game.

Iterative deepening as a safety mechanism: because alpha-beta always retains the best action from the last fully completed depth, it never returns an incomplete result. If the deadline arrives mid-search at depth 5, the agent returns the depth-4 result rather than a partial depth-5 result, making the search safe under tight time constraints.

Dual evaluation functions: alpha-beta and MCTS use different evaluation functions suited to their roles. The alpha-beta `evaluate()` function returns values in $[-1, 1]$ and includes centre control, which matters more in the structured early game. The MCTS `rollout_score()` returns values in $[0, 1]$ and adjusts weight parameters according to end game specifications, i.e. placing less importance on distance to the nearest enemy as it is redundant.